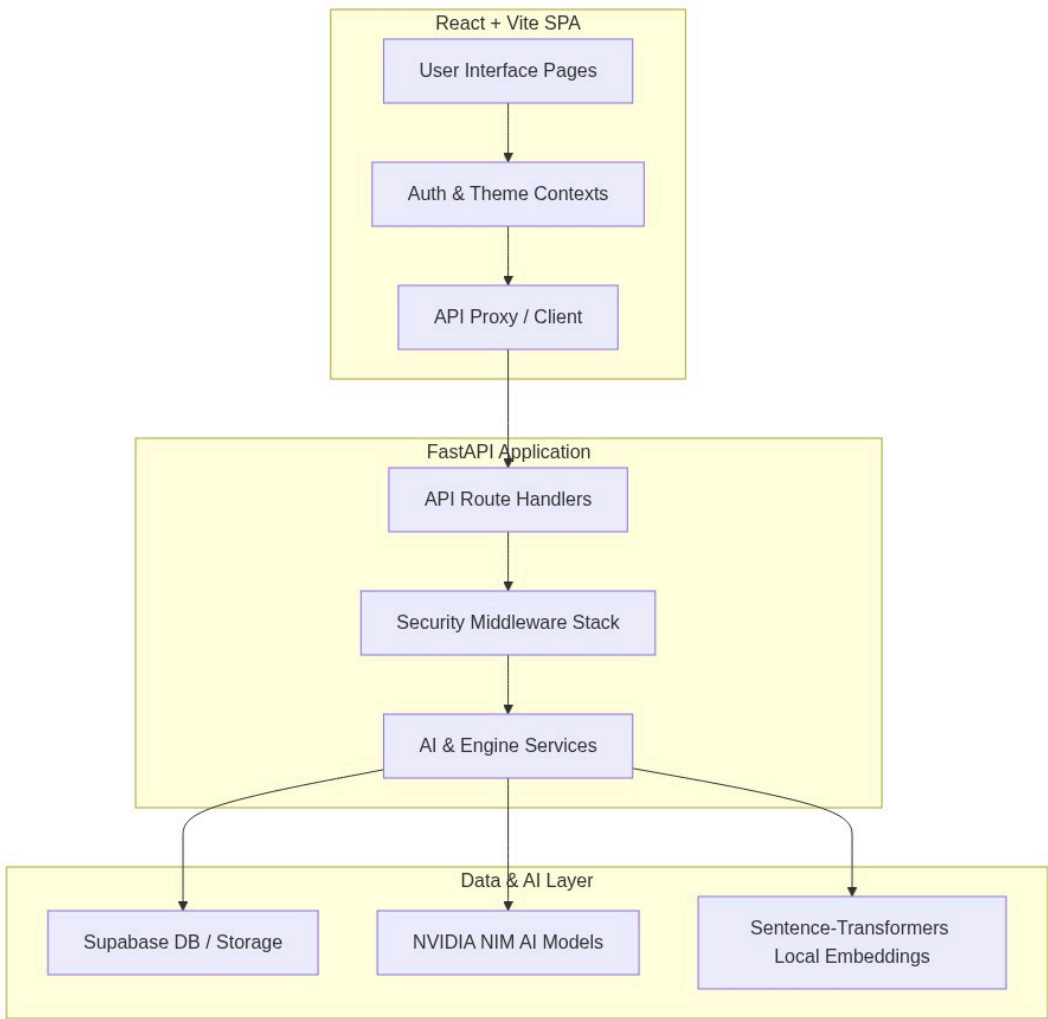


Project Analysis: HireSense ATS

HireSense is an AI-powered Applicant Tracking System (ATS) that helps recruiters parse resumes, match candidates against job descriptions (JDs) with semantic precision, check ATS compatibility, and manage the hiring pipeline using a cinematic and responsive interface.

Architecture & Component Design

The project is structured as a full-stack web application with a decoupled React frontend and a FastAPI backend:



Technology Stack & Core Layers

1. Frontend

- Framework: React (v19) configured with Vite for building a single-page application.
- Styling: Tailwind CSS for responsive layouts and Vanilla CSS variable-based styling (index.css) defining

light/dark modes.

- State Management: React Contexts for global authentication (AuthContext) and theme options (ThemeContext).
- Networking: fetch API-based client proxy mapping client components to backend endpoints.

2. Backend

- Framework: FastAPI (Python 3.11+) implementing async endpoints and dependency injection.
- Servers: Uvicorn server configuration with built-in CORS, security, and payload restrictions.
- ORM & DB Connection: supabase-py client library interacting with REST endpoints.

3. AI & Matching Engines

- NVIDIA NIM Racing Engine: Concurrently queries multiple models (llama-3.1-8b-instruct, llama-3.2-3b-instruct, llama-3.2-1b-instruct) using the openai async SDK, adopting a "fastest valid response wins" approach.
- Embedding Engine: Local loading of Sentence-Transformers all-MiniLM-L6-v2 generating 384-dimensional dense vector embeddings.
- Heuristic Engines: Python regex and text processors analyzing years of experience, keyword densities, resume strength, and potential bias indicators.

Database & Schema Design

The persistence tier relies on a Supabase PostgreSQL database. It utilizes pgvector for similarity calculations and has disabled Row Level Security (RLS) to delegate access control strictly to the backend API layer.

Core Tables

``resumes``

Stores candidate profiles and parsed resume metadata:

- id: UUID (Primary Key).
- user_id: TEXT (maps ownership to Supabase Auth UUID).
- file_url: TEXT (path proxy for private storage downloads).
- raw_text: TEXT (normalized parsed content).
- candidate_name: TEXT (extracted candidate name).
- embedding: VECTOR(384) (calculated text embeddings).
- ats_score: FLOAT (calculated ATS compliance rating).
- ats_breakdown: JSONB (detailed recommendations checklist).
- status: TEXT (processing, completed, or failed state).

``job_descriptions``

Contains target profiles uploaded by recruiters:

- id: UUID (Primary Key).

- `user_id`: TEXT (identifies resource owner).
- `job_text`: TEXT (JD content).
- `title`: TEXT (job title).
- `embedding`: VECTOR(384) (384-dimensional semantic embedding).
- `jd_hash`: TEXT (de-duplication hash).

``match_results``

Represents candidate-to-job matches and evaluations:

- `id`: UUID (Primary Key).
- `user_id`: TEXT (resource owner identifier).
- `resume_id`: UUID (references resumes).
- `job_id`: UUID (references job_descriptions).
- `semantic_score`: FLOAT (cosine similarity score between embeddings).
- `skill_score`: FLOAT (overlap ratio of taxonomical skills).
- `experience_score`: FLOAT (experience years assessment).
- `final_score`: FLOAT (weighted aggregate score).
- `resume_strength`: INT (profile completeness metric).
- `risk_level`: TEXT (Low, Medium, or High classification).
- `fair_mode_enabled`: BOOLEAN (toggle to skip direct years-of-experience weighting).
- `bias_report`: JSONB (demographic flag alerts).
- `candidate_status`: TEXT (recruitment stages: pending, approved, or rejected).

``profiles``

Recruiter profile settings linked to Supabase Auth:

- `id`: UUID (Primary Key referencing `auth.users(id)`).
- `display_name`: TEXT (custom username).
- `avatar_url`: TEXT (bucket link for public profile picture).

``auth_logs``

System-wide security auditing table:

- `id`: UUID (Primary Key).
- `user_id`: UUID (authenticated actor).
- `email`: TEXT (quick visibility).
- `event_type`: TEXT (login, logout, signup trails).
- `ip_address`, `user_agent`: Metadata parameters.

Security Features & Data Isolation

1. JWT Authentication: All secure endpoints are gated using a shared dependency (`require_user`) validating the header's Bearer token via Supabase Auth.

2. Resource Scoping: Strict logical isolation is implemented at the SQL query level where all read, write, update, and delete actions filter by `user_id` matches.

3. Admin Controls: Master panel commands (`routes/admin.py`) are protected using `require_admin` which checks for designated email authorization (`aditya.poddar3698@gmail.com`) to allow system auditing and user operations.

4. Middleware Protection:

- Request Size Limiter: Rejects requests larger than 10MB to prevent denial-of-service attempts.
 - Response Security Headers: Sets HTTP standards (`X-Content-Type-Options`, `X-Frame-Options`, `X-XSS-Protection`, etc.).
 - Safe File Proxy: Sanitizes filename patterns in the upload downloader endpoint to block path traversal vulnerabilities.
-

AI Engines & Decision Workflows

1. Multi-Model LLM Racing

The racing engine (`services/llm_service.py`) tackles latency and rate-limiting bottlenecks using an asynchronous concurrent design:

- Fires concurrent network requests to NVIDIA NIM endpoints.
- Monitors three target models in parallel:
 1. Primary: Llama-3.1-8B (15s cutoff)
 2. Speed Runner: Llama-3.2-3B (10s cutoff)
 3. Backup Runner: Llama-3.2-1B (8s cutoff)
- Automatically captures the fastest returning valid JSON schema.
- Employs an in-memory caching wrapper (`_llm_cache`) that maps hashed prompt payloads to instant results.
- Automatically falls back to a heuristic mock generator if all API requests fail.

2. Matching Formula

The evaluation pipeline combines four distinct sub-scores:

- Semantic Score (40%): Cosine similarity between dense Sentence-Transformer embedding vectors.
- Skill Score (30%): Taxonomy overlap matching parsed text against standard terms.
- Experience Fit (20%): Years of experience matching algorithm.
- Resume Strength (10%): Profile completeness based on character length and numeric metrics.

Note: Enabling Fair Mode shifts the matching weights to 55% Semantic Score, 35% Skill Score, and 10% Resume Strength (completely ignoring the direct experience score metric).